

06 — Controllability

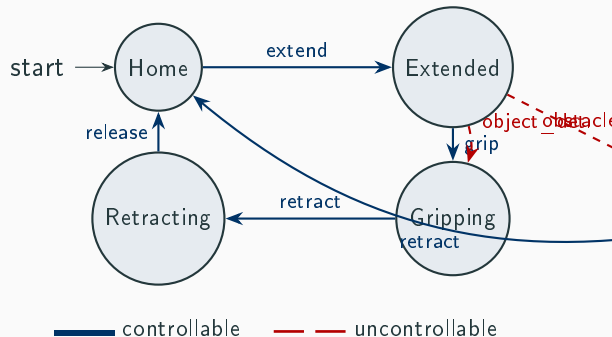
Controllable vs. Uncontrollable Actions

Mariano Cerrutti

Mununu Tutorial

The Concept

- **Controllable** — actions the controller *decides*: extend, retract, grip, release.
- **Uncontrollable** — environment events that *cannot be blocked*: object_detected, obstacle_detected.



Declaring Controllable Labels

```
1      automaton RobotArm {
2          // Only actuator commands are controllable.
3          // Sensor inputs (object_detected, obstacle_detected) are
4          // uncontrollable --the environment triggers them.
5          controllable {
6              label extend;
7              label retract;
8              label grip;
9              label release;
10         }
11
12         states {
13             state Home initial;
14             state Extended;
15             state Gripping;
16             state Retracting;
17             state Blocked;
18         }
```

Robot Arm Transitions

```
1      transitions {
2          // Normal operation cycle
3          transition Home -> Extended on label extend;
4          transition Extended -> Gripping on label grip;
5          transition Gripping -> Retracting on label retract;
6          transition Retracting -> Home on label release;
7
8          // Object detection triggers grip (environment event)
9          transition Extended -> Gripping on label object_detected;
10
11         // Obstacle can appear while extended (environment event)
12         transition Extended -> Blocked on label obstacle_detected;
13         // Must retract when blocked
14         transition Blocked -> Home on label retract;
15
16         // Idle self-loops
17         transition Home -> Home on label idle;
18         transition Extended -> Extended on label idle;
```

Controllable Reachability: Return Home

Can the controller *force* returning to **Home** from every state?

```
1 // despite uncontrollable obstacle_detected/object_detected events?
2 // [ (ctrl = controllable) ] X ensures:
3 // - all uncontrollable transitions stay in the winning region
4 // - at least one controllable transition progresses toward Home
5 formula ctrl_return_home {
6     over RobotArm;
7     body = mu X. (Home || [ (ctrl = controllable) ] X);
8 }
```

- [(ctrl = controllable)] X ensures:
 - All uncontrollable transitions stay in the winning region.
 - At least one controllable transition progresses toward Home.
- **Result:** 4/5 states satisfy — **Extended** is excluded because `idle` is uncontrollable there, letting the environment stall indefinitely.
- The controller cannot force progress from **Extended** when the environment can

Controller Synthesis

```
1  controllers {  
2      // Synthesize a controller that enforces safety.  
3      // The controller can only restrict controllable actions;  
4      // uncontrollable environment events always pass through.  
5      controller safe_robot {  
6          source RobotArm;  
7          satisfying safety_invariant;  
8      }  
9  }
```

CLI invocation:

```
$ mununu context synth 06_controllability.ctxdsl \  
  --formula safety_invariant --automaton RobotArm
```

- `source` — the automaton to restrict.
- `satisfying` — the property the controller must enforce.

Synthesis Result and Key Rules

Result: the controller retains all 5 states — the property is **realizable** without removing any state.

How synthesis works:

1. Compute the *winning region* — states from which the property can be enforced despite the environment.
2. *Restrict* controllable transitions that leave the winning region.
3. *Preserve* all uncontrollable transitions (environment cannot be blocked).

Key rule for composition:

Controllable labels must **not** be shared between automata in a composition — each controller owns its own actions.